

CampusConnect

Smart College Management & Collaboration Platform

Document Type: Full Technical Architecture & Guide

Tech Stack: MERN (MongoDB, Express, React, Node)

Author: CampusConnect Core Team

Status: Production Ready & Verified

Date: August 2026

Version: 2.0.0

1. Executive Summary & Project Purpose

CampusConnect is an enterprise-grade, role-based Smart College Management & Collaboration Platform built using the MERN stack. It establishes a seamless real-time ecosystem connecting **Students**, **Faculty Members**, and **College Administrators**. The platform digitizes academic structures, date-wise subject attendance, automated student notifications, academic analytics, assignment submissions, announcements, grievance redressal, and campus events.

2. Core Architecture & System Workflows

The system is designed around 4 fundamental real-world academic workflows:

- **A. Permanent Subject & Faculty Mapping:** Faculty creates a subject ONCE for a specific academic class (e.g. IT-D1). That subject permanently belongs to that faculty member and remains accessible under 'My Subjects'.
- **B. Subject-Based Date-Wise Attendance:** Faculty takes attendance for their assigned subject date-by-date. If attendance was already recorded for a selected date, the system automatically enters **Edit Mode**, allowing faculty to update existing records.
- **C. Instant In-App Student Notifications:** Immediately after faculty submits or updates attendance, MongoDB triggers in-app notifications (`Notification` model) for all rostered students, displaying their attendance status (Present/Absent).
- **D. Dynamic Student Percentage & No-Lecture Protection:** Calculates real-time attendance percentage per subject and overall. If total sessions equal 0, the system displays a neutral 'No Lectures' rating instead of defaulting to 100%.

3. Backend File-by-File & Function Breakdown (`server/`)

File Name	Responsibility & Functions Executed
<code>server.js</code>	Entry point of Express server. Configures CORS, JSON body parser middleware, connects to MongoDB Atlas via <code>`connectDB()`</code> , registers API route handlers (<code>`/api/auth`</code> , <code>`/api/departments`</code> , <code>`/api/academic`</code> , <code>`/api/attendance`</code> , <code>`/api/notifications`</code> , etc.), and handles global errors.
<code>config/db.js</code>	Handles MongoDB Atlas connection using Mongoose. Includes automatic DNS lookup fallback (<code>`8.8.8.8`</code>) for Windows network SRV query issues, connection status logging, and local MongoDB fallback.
<code>middleware/authMiddleware.js</code>	Provides <code>`protect`</code> middleware to verify JWT tokens from <code>`Authorization: Bearer`</code> headers, and <code>`authorize(...roles)`</code> middleware to enforce strict Role-Based Access Control (RBAC).
<code>models/User.js</code>	Defines User schema (roles: <code>`admin`</code> , <code>`faculty`</code> , <code>`student`</code>). Encrypts passwords using <code>`bcryptjs`</code> pre-save hooks and provides <code>`matchPassword()`</code> method.
<code>models/Department.js</code>	Stores college departments (e.g. <code>`Computer Science`</code> , <code>`Information Technology`</code> , <code>`AIDS`</code> , <code>`ECS`</code> , <code>`EXTC`</code>).
<code>models/AcademicClass.js</code>	Stores division/class structures (e.g. <code>`IT-D1`</code> , <code>`CS-D2`</code>). Linked to <code>`Department`</code> via Mongo ObjectId.
<code>models/StudentEnrollment.js</code>	Maps students to their respective academic class and roll number (<code>`01`</code> to <code>`70`</code>).
<code>models/Subject.js</code>	Defines academic subjects. Stores <code>`name`</code> , <code>`code`</code> , <code>`faculty`</code> (User ObjectId), <code>`academicClass`</code> (AcademicClass ObjectId), and <code>`department`</code> .
<code>models/AttendanceSession.js</code>	Represents a single lecture session taken on a specific date (<code>`date`</code> , <code>`sessionTime`</code> , <code>`subject`</code> , <code>`academicClass`</code> , <code>`faculty`</code>).
<code>models/AttendanceRecords.js</code>	Stores individual student attendance status (<code>`student`</code> , <code>`attendanceSession`</code> , <code>`subject`</code> , <code>`academicClass`</code> , <code>`status: Present/Absent`</code>).
<code>models/Notification.js</code>	Stores student and faculty in-app alerts (<code>`recipient`</code> , <code>`title`</code> , <code>`message`</code> , <code>`type: attendance/event/etc.`</code> , <code>`isRead`</code>).
<code>controllers/attendanceController.js</code>	Contains business logic for attendance management: <code>`getStudentsForSession`</code>: Fetches class roster and checks if a session already exists for the selected date (Edit Mode). <code>`recordAttendanceSession`</code>: Saves session & records, then dispatches <code>`Notification`</code> documents to all students. <code>`updateAttendanceSession`</code>: Updates existing attendance records and dispatches update notifications. <code>`getStudentAttendanceSummary`</code>: Calculates per-subject and overall attendance percentages with zero-session safety. <code>`getFacultyAttendanceHistory`</code>: Retrieves historic attendance logs for faculty subjects.
<code>controllers/academicController.js</code>	Manages subject creation, listing, subject deletion, and academic class retrieval (<code>`createSubject`</code> , <code>`getMySubjects`</code> , <code>`getSubjectsForClass`</code>).

<code>controllers/authController.js</code>	Handles <code>`registerUser`</code> , <code>`loginUser`</code> , and <code>`getMe`</code> user profile retrieval.
<code>controllers/departmentController.js</code>	Handles department creation, listing, and deletion by administrators.
<code>controllers/userController.js</code>	Manages student and faculty account listings, role updates, and user deletions.

4. Frontend File-by-File & UI Component Breakdown (``client/``)

Component / Page Path	UI Responsibilities & Workflow
<code>src/App.jsx</code> & <code>AppRoutes.jsx</code>	Configures React Router v6 routes. Wraps admin, faculty, and student pages inside <code>`ProtectedRoute`</code> guards checking user JWT role.
<code>src/components/Navbar.jsx</code>	Top navigation bar. Renders user avatar, role badge, and embeds <code>`NotificationDropdown.jsx`</code> .
<code>src/components/NotificationDropdown.jsx</code>	Real-time notification bell <code>`🔔`</code> . Polls unread count, displays attendance alerts with custom icons, and handles mark-as-read.
<code>src/components/Sidebar.jsx</code>	Navigation drawer dynamically filtering links based on logged-in user role (<code>`admin`</code> , <code>`faculty`</code> , <code>`student`</code>).
<code>src/pages/LandingPage.jsx</code>	Public landing page featuring platform statistics, 3D Hero canvas, features list, and direct portal entry buttons.
<code>src/pages/faculty/TakeAttendancePage.jsx</code>	Faculty attendance interface. Selects date, auto-detects Edit Mode if attendance exists for selected date, toggles Present/Absent per student, and submits batch attendance.
<code>src/pages/faculty/FacultySubjectsPage.jsx</code>	My Subjects dashboard. Allows faculty to create permanent subjects once for assigned classes.
<code>src/pages/student/StudentAttendancePage.jsx</code>	Student attendance dashboard. Displays overall percentage, subject-wise attendance breakdown, status rating badges (<code>`'No Lectures'`</code> , <code>`'Good'`</code> , <code>`'Low Attendance'`</code>), and exports printable PDF report.
<code>src/pages/AnalyticsPage.jsx</code>	Platform Analytics page rendering Recharts BarChart (<code>`'Students by Department'`</code>) with collision-free angled labels and PieChart (<code>`'User Role Distribution'`</code>).
<code>src/pages/admin/*</code>	Admin portal pages for Department Management, Class Management, Subject Management, Student Enrollment, and Activity Logs.

5. Live Academic Department & Student Roster Summary

Department Name	Code	Faculty Count	Division 1 (D1)	Division 2 (D2)	Total Students
Information Technology	IT	3 Faculty	36 Students	35 Students	71 Students
Computer Science	CS	3 Faculty	50 Students	70 Students	120 Students
AI & Data Science	AIDS	3 Faculty	20 Students	30 Students	50 Students
Electronics & Computer Science	ECS	3 Faculty	20 Students	30 Students	50 Students
Electronics & Telecommunication	EXTC	3 Faculty	20 Students	30 Students	50 Students

6. Deployment & Environment Status

- **Backend API:** Live on Render (``campusconnect-backend.onrender.com``).
- **Database:** Live MongoDB Atlas Cluster with SRV lookup & DNS safety.
- **GitHub Repository:** ``https://github.com/Shubham46-glitch/CampusConnect.git`` (Branch: ``main``).
- **Build Verification:** Vite production build passes in ~7 seconds with 0 errors.